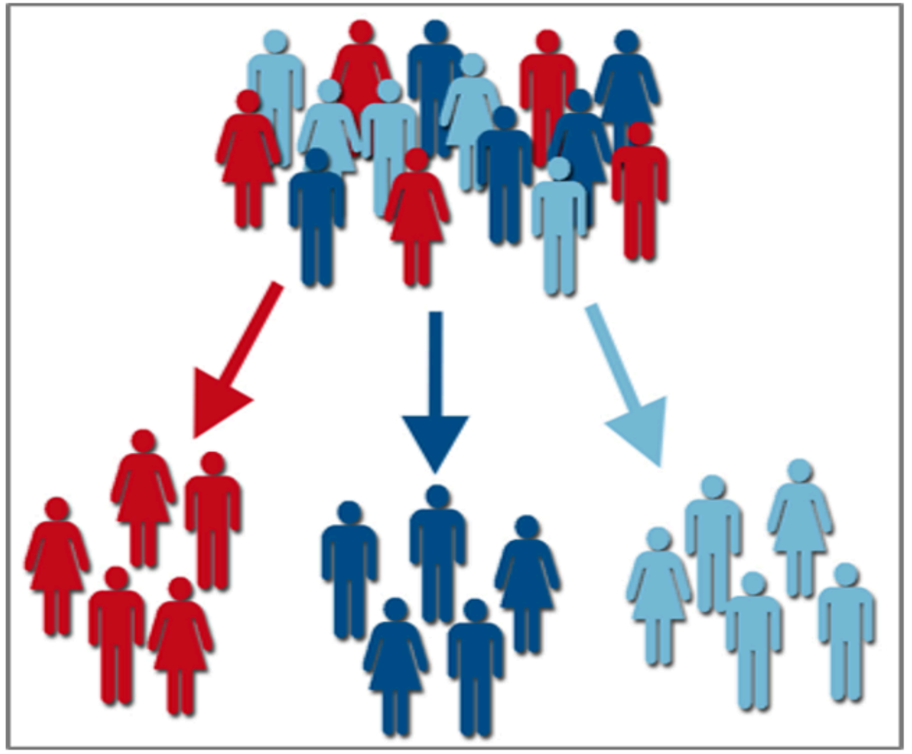
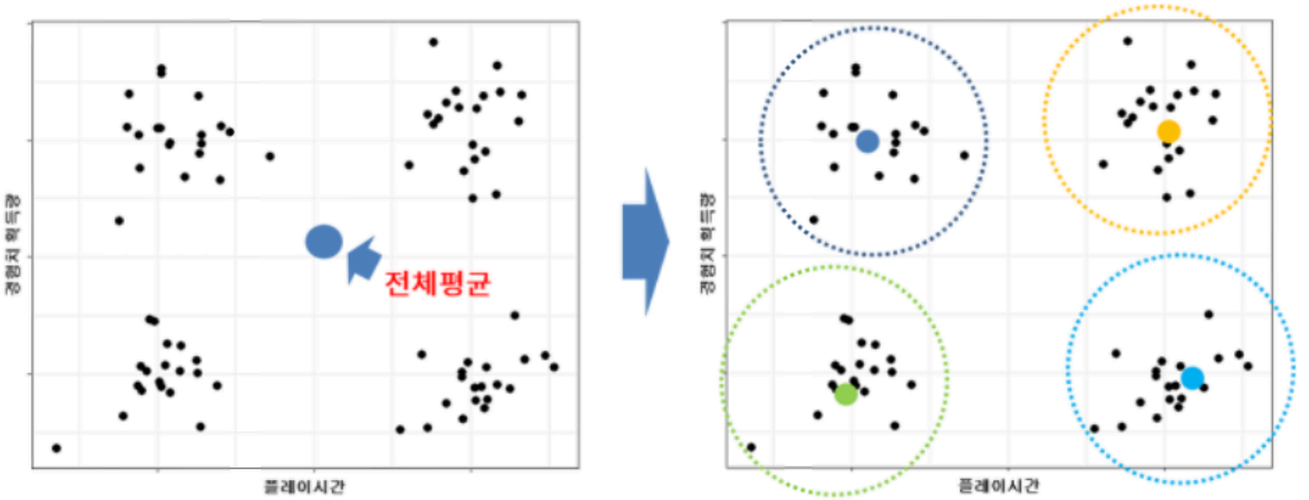
AUTOENCODER

Unsupervised Learning: Clustering

비슷한 속성을 가진 군집을 묶어내기





비슷한 속성을 가진 군집을 어떻게 묶어낼까?

https://danbi-ncsoft.github.io/works/2018/06/28/works-game-user-segmentation-1.html

Unsupervised Learning: Clustering

비슷한 속성을 가진 군집을 어떻게 묶어낼까?

실제 사례

- 소비 패턴에 따라 고객 유형 나누기
- 유사한 토픽끼리 뉴스기사 분류하기



Credit: https://www.thinkwithgoogle.com/intl/ko-kr/marketing-strategies/data-and-measurement/%EB%B6%88%ED%99%95%EC%8B%A4%EC%84%B1%EC%9D%98-%EC%8B%9C%EB%8C%80%EC%97%90-%EB%AF%BC%EC%B2%A9%ED%95%98%EA%B2%8C-%EB%8C%80%EC%9D%91%ED%95%98%EB%8A%94-%EA%B8%B0%EC%97%85%EB%93%A4%EC%9D%98-%EB%8D%B0%EC%9D%B4%ED%84%B0-%ED%99%9C%EC%9A%A9-%EC%82%AC%EB%A1%80/

Unsupervised Learning: Clustering

'비슷하다'는 것이 무엇일까?

test image					training image					pixel-wise absolute value differences				
56	32	10	18	-	10	20	24	17	=	46	12	14	1	→ 456
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200		12	16	178	170		12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	

' **유사도(거리)' 기준**이 필요하다!

- L1 norm
- L2 norm

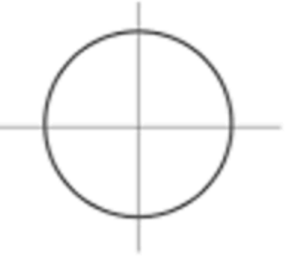
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



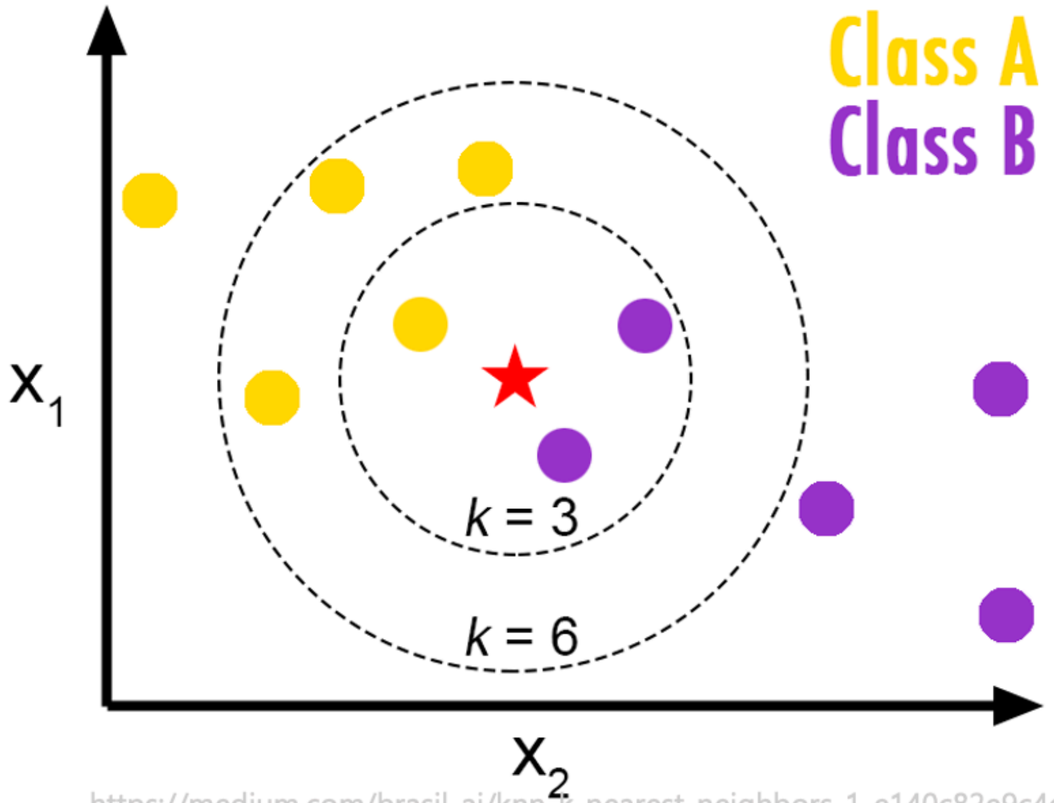
L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



Unsupervised Learning: Clustering

비슷한 속성을 가진 군집을 어떻게 묶어낼까?



https://medium.com/brasil-ai/knn-k-nearest-neighbors-1-e140c82e9c4e

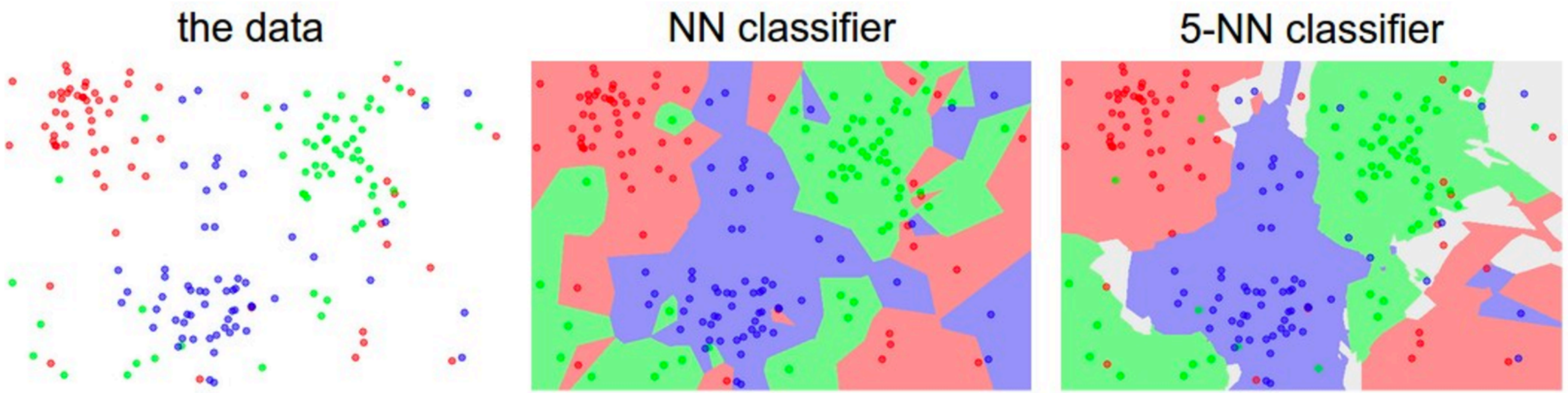
K-Nearest Neighbor Classifier

주변(이웃)에서
더 많은 데이터가 포함되어 있는 범주로 분류하는 방식

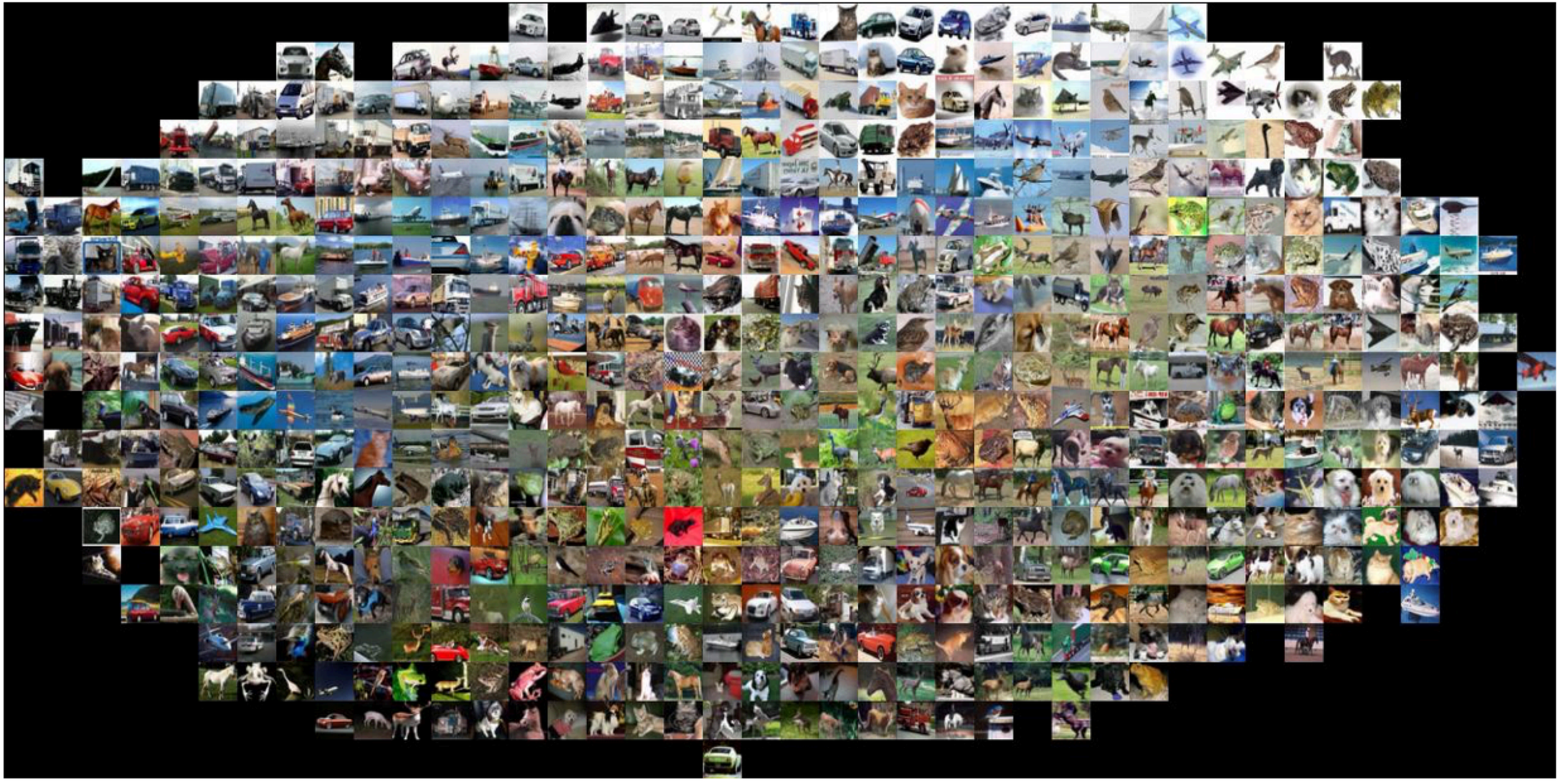
Unsupervised Learning: Clustering

비슷한 속성을 가진 군집을 어떻게 묶어낼까?

K-Nearest Neighbor Classifier

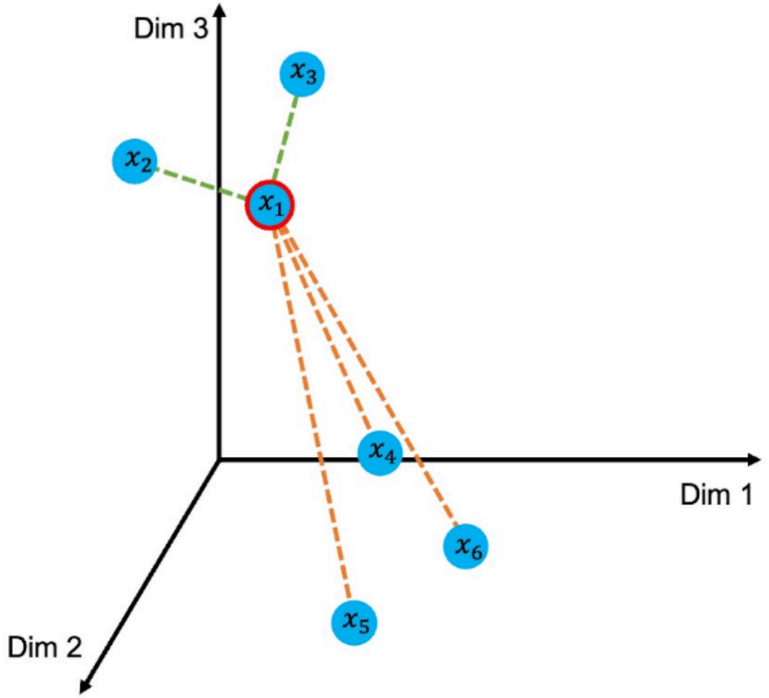


차원 축소 모델 ‘t-SNE’로 군집화한 CIFAR-10

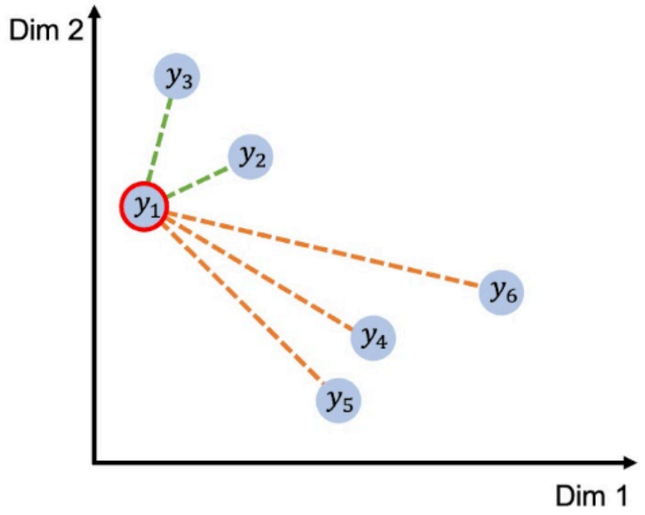


데이터의 유사성을 기반으로, 2차원에 군집화하여 나타낸 것

t-SNE



고차원의 원래 데이터



저차원으로 축소된 데이터

고차원에서도 거리는 측정이 되니까...

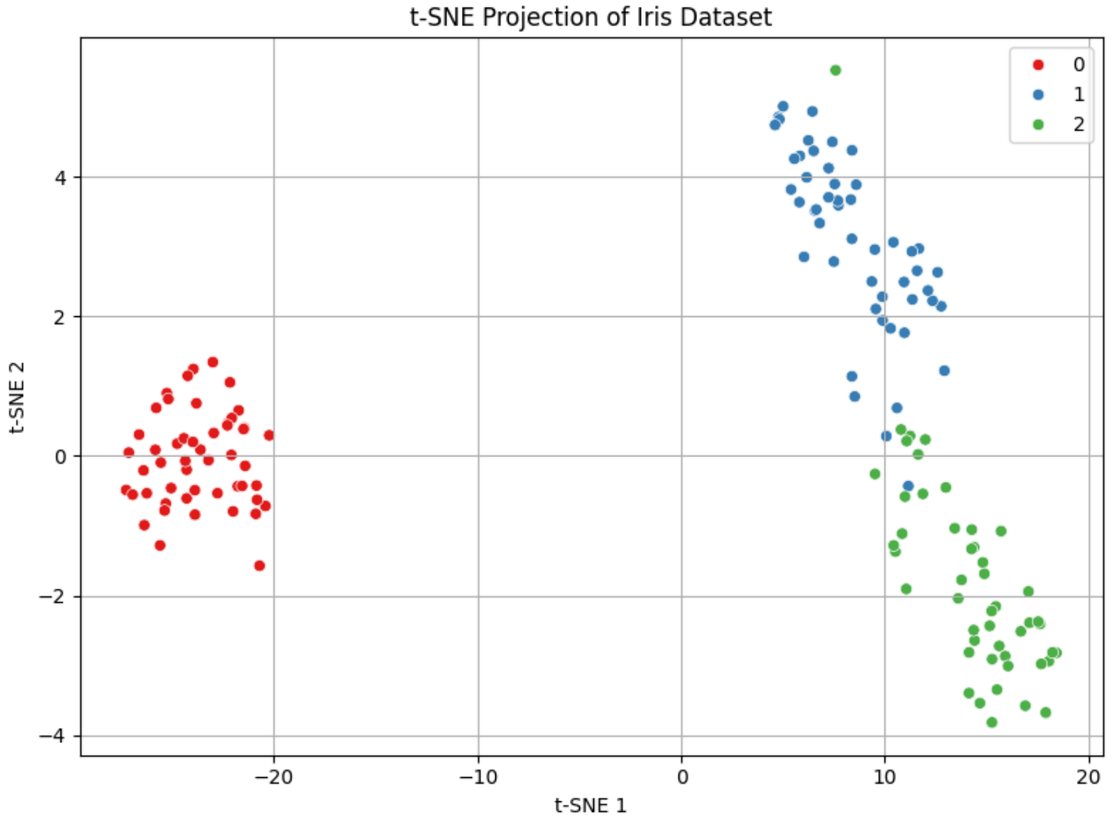
저차원에서 데이터간 거리를 유사하게 표현할 수 있는 분포를 열심히 찾아보면 어떨까?

```python
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns

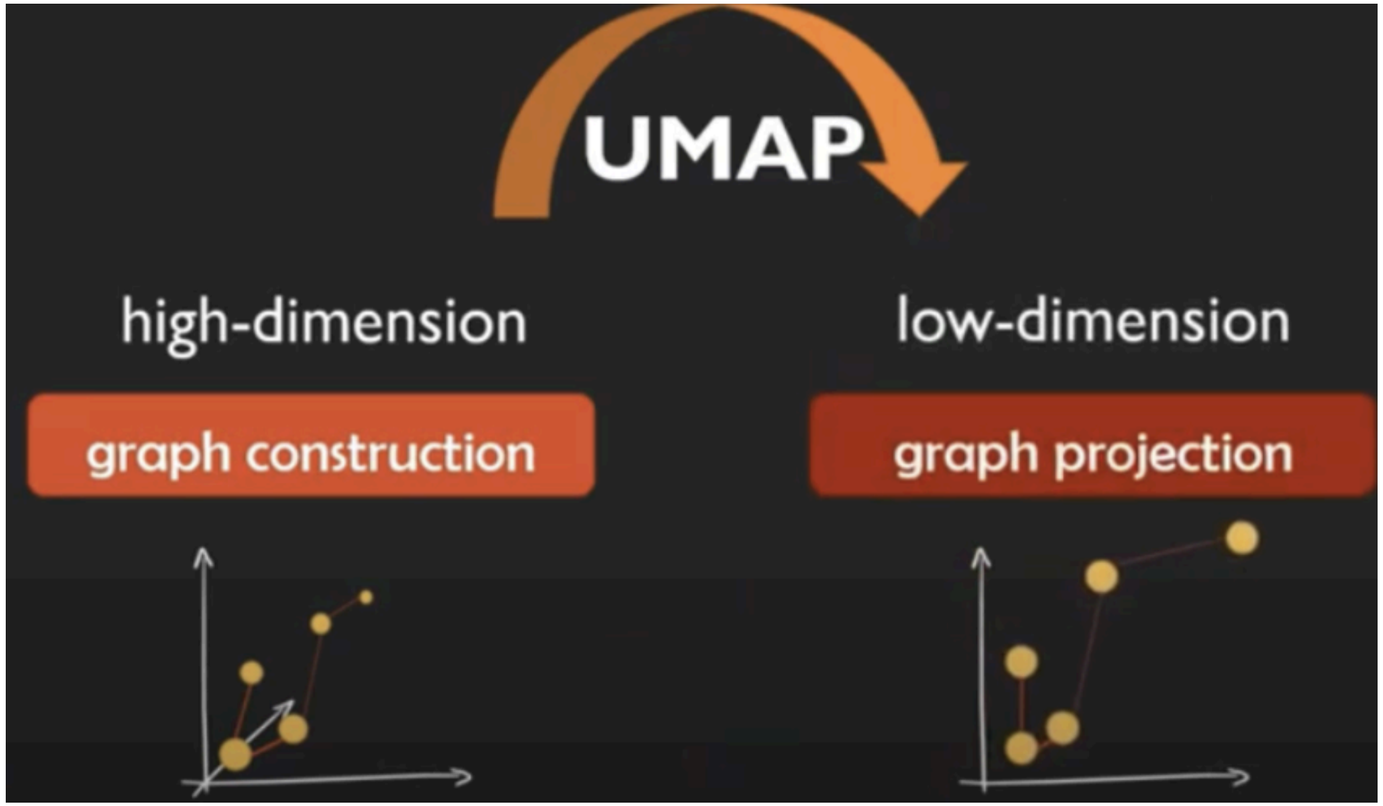
# 1. 데이터 로드
iris = load_iris()
X = iris.data
y = iris.target

# 2. t-SNE 임베딩 (2차원)
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

# 3. 시각화
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y, palette='Set1', legend='full')
plt.title("t-SNE Projection of Iris Dataset")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.grid(True)
plt.tight_layout()
plt.show()
```



umap



그걸 그래프 이론을 이용해서 수학적으로 더 좋은 방법으로 차원을 줄이면 어떨까?

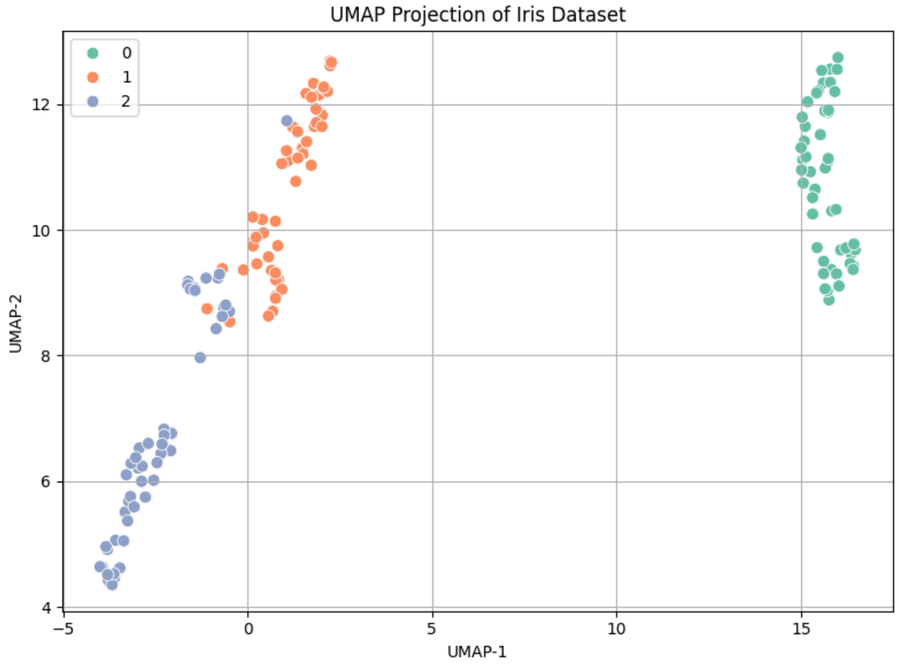
```
!pip install umap-learn
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import umap
from sklearn.datasets import load_iris

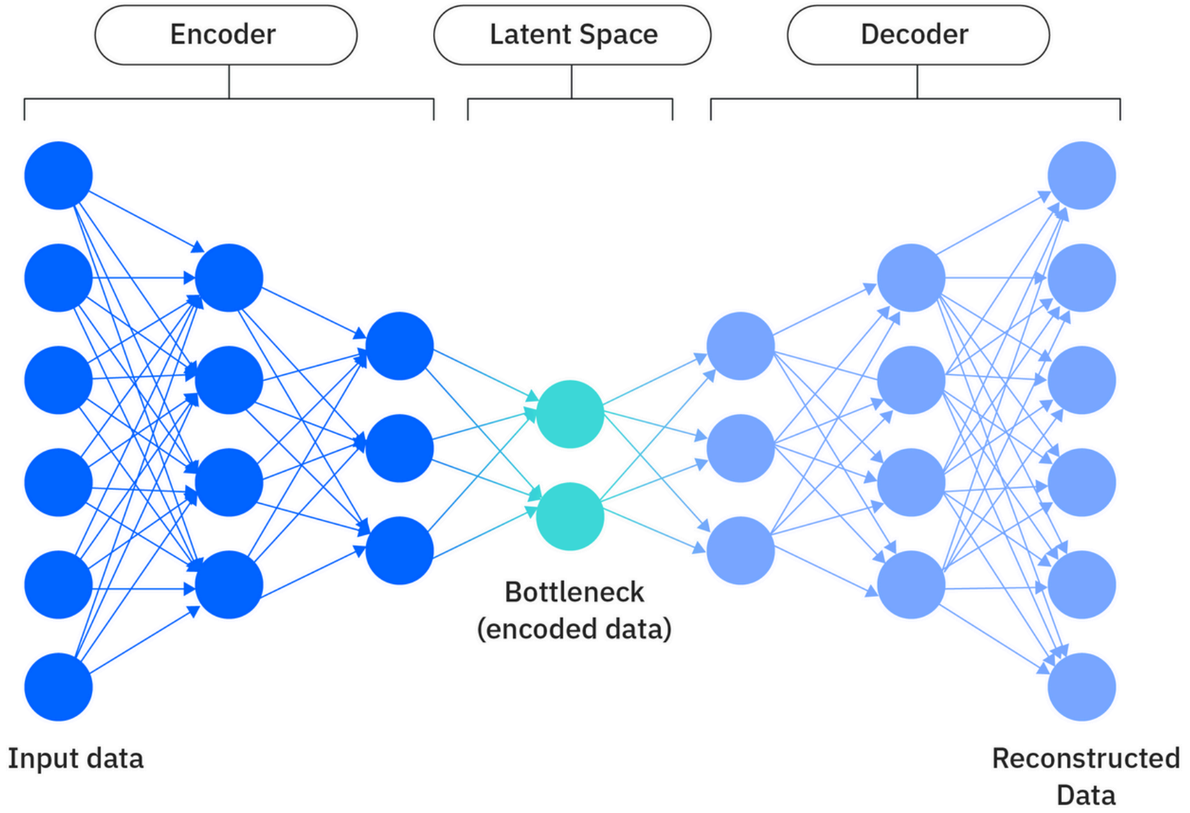
# 1. 데이터 로드
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# 2. UMAP 임베딩 (2차원)
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
X_umap = reducer.fit_transform(X)

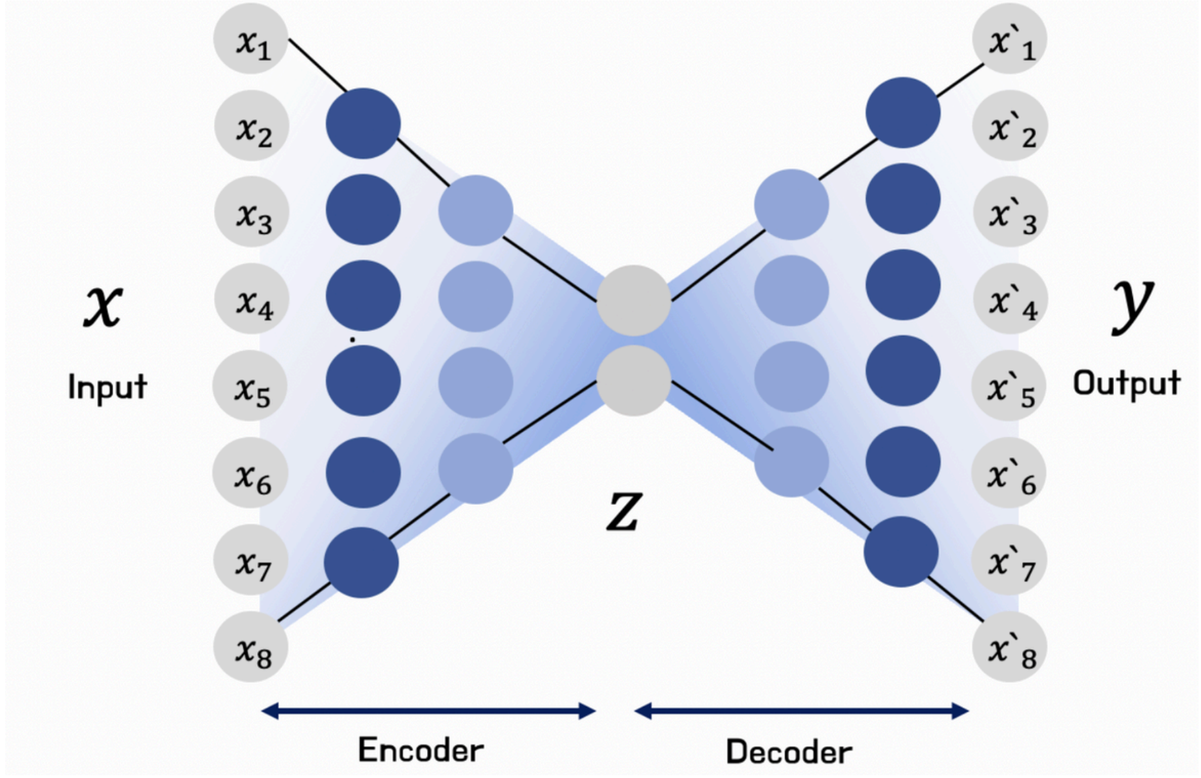
# 3. 시각화
plt.figure(figsize=(8, 6))
sns.scatterplot(x=X_umap[:, 0], y=X_umap[:, 1], hue=y, palette='Set2', s=60)
plt.title("UMAP Projection of Iris Dataset")
plt.xlabel("UMAP-1")
plt.ylabel("UMAP-2")
plt.grid(True)
plt.tight_layout()
plt.show()
```



auto-encoder



차원을 줄인다면 이렇게 줄일수는 없을까?




```python
# 오토인코더 모델 정의
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 12),
            nn.ReLU(),
            nn.Linear(12, 3)  # 압축된 표현
        )
        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(3, 12),
            nn.ReLU(),
            nn.Linear(12, 64),
            nn.ReLU(),
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, 28*28),
            nn.Sigmoid()  # 픽셀 각각이 0과 1 사이의 값을 가져야 하므로
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded
```

```python
# 데이터 로딩
transform = transforms.Compose([
    transforms.ToTensor(),  # 텐서로 변환
    transforms.Normalize((0.5,), (0.5,))  # 정규화
])
train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
train_loader = DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)

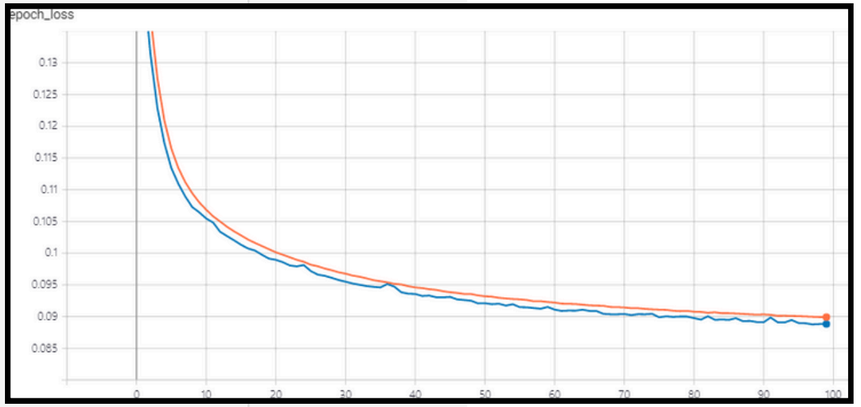
# 모델, 손실 함수 및 옵티마이저 초기화
model = Autoencoder()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 훈련 과정
num_epochs = 20
for epoch in range(num_epochs):
    for data in train_loader:
        img, _ = data  # 레이블은 사용하지 않습니다.
        img = img.view(img.size(0), -1)
        # 순전파
        output = model(img)
        loss = criterion(output, img)

        # 역전파
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

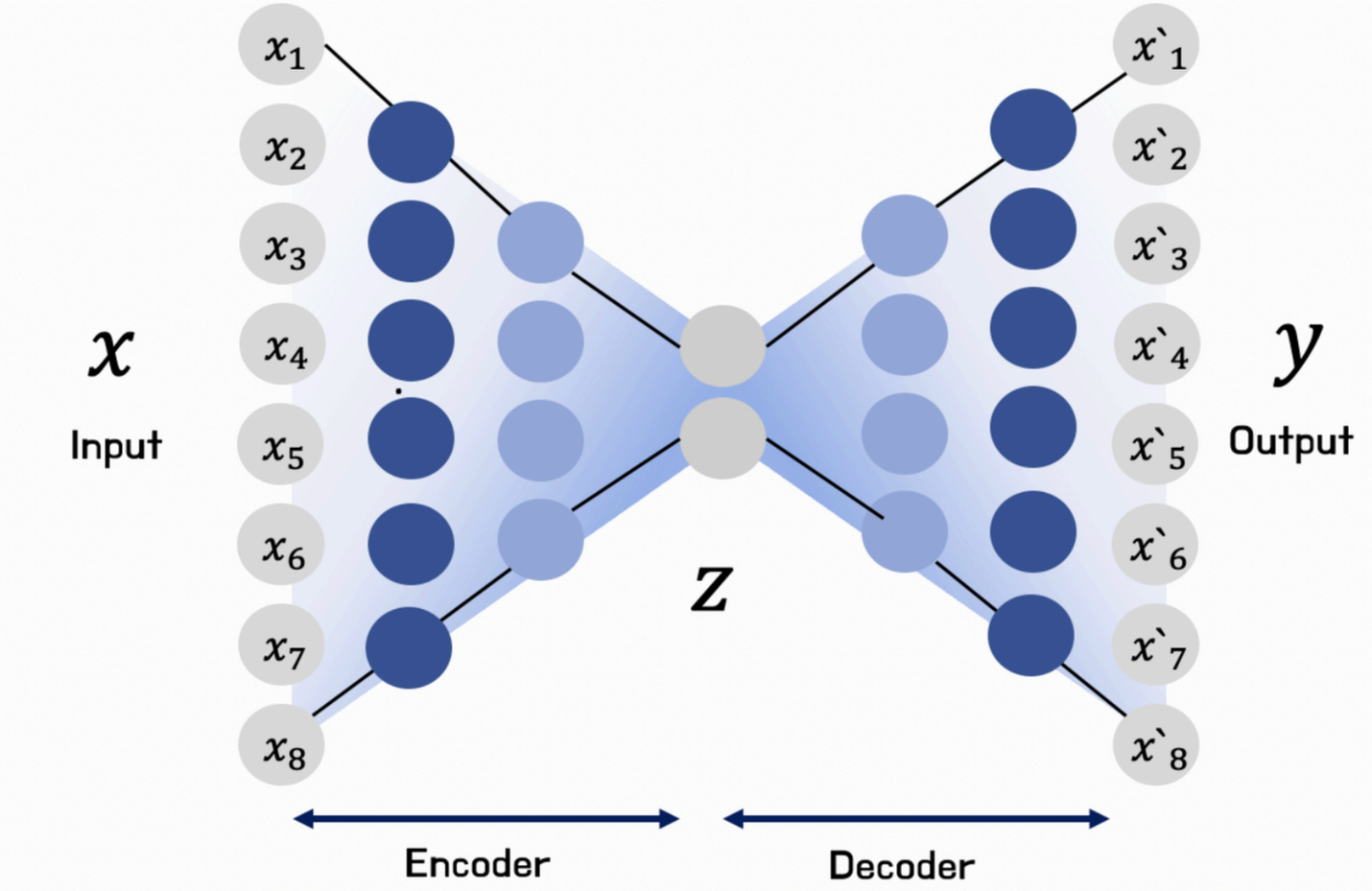
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

print("Training completed")
```



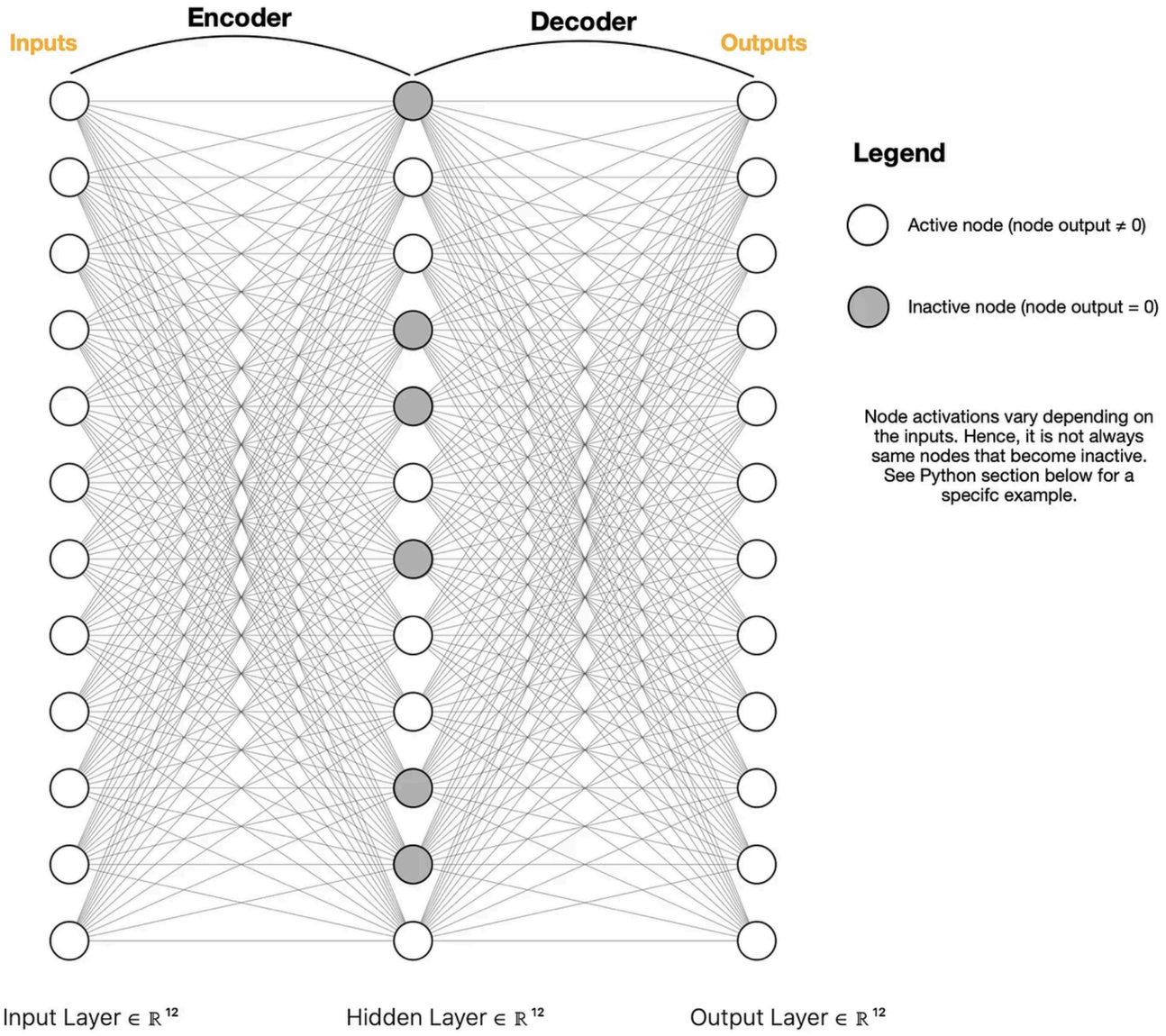


Z는 그래서 뭔가요?



액기스?

Sparse Autoencoder



입력 데이터에서 불필요한 정보를 제거,

의미 있는 특징만 표현하는 압축!!

실제 뉴런처럼...

잠재 벡터를 sparse(희소하게) 만들자

```python
# Initialize the Sparse Autoencoder, Loss function, and Optimizer
model = Autoencoder()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop with sparsity penalty
num_epochs = 20
sparsity_lambda = 0.001  # Weight for the L1 sparsity penalty

for epoch in range(num_epochs):
  for data in train_loader:
    img, _ = data
    img = img.view(img.size(0), -1)  # Flatten the images
    recon, encoded = model(img)
    recon_loss = criterion(recon, img)

    # L1 sparsity penalty on the encoded layer
    sparsity_loss = torch.mean(torch.abs(encoded))

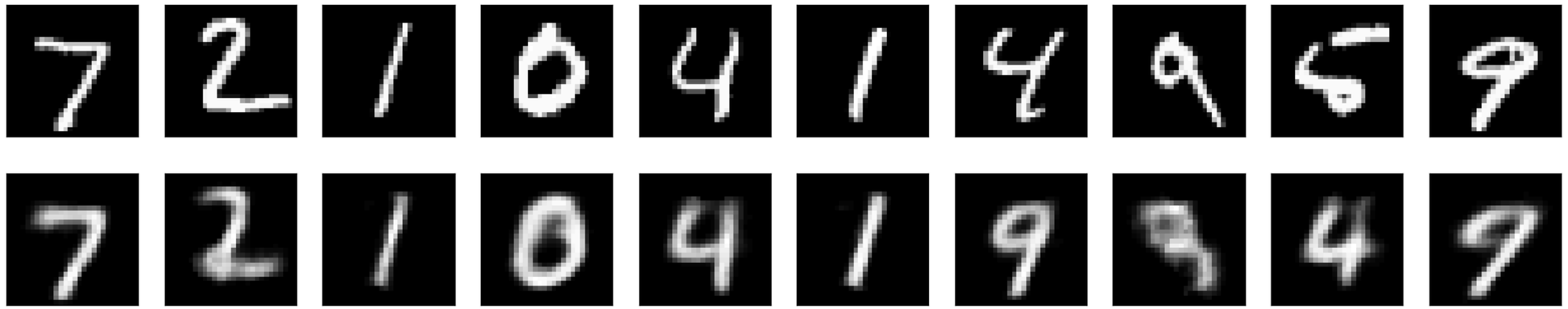
    loss = recon_loss + sparsity_lambda * sparsity_loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

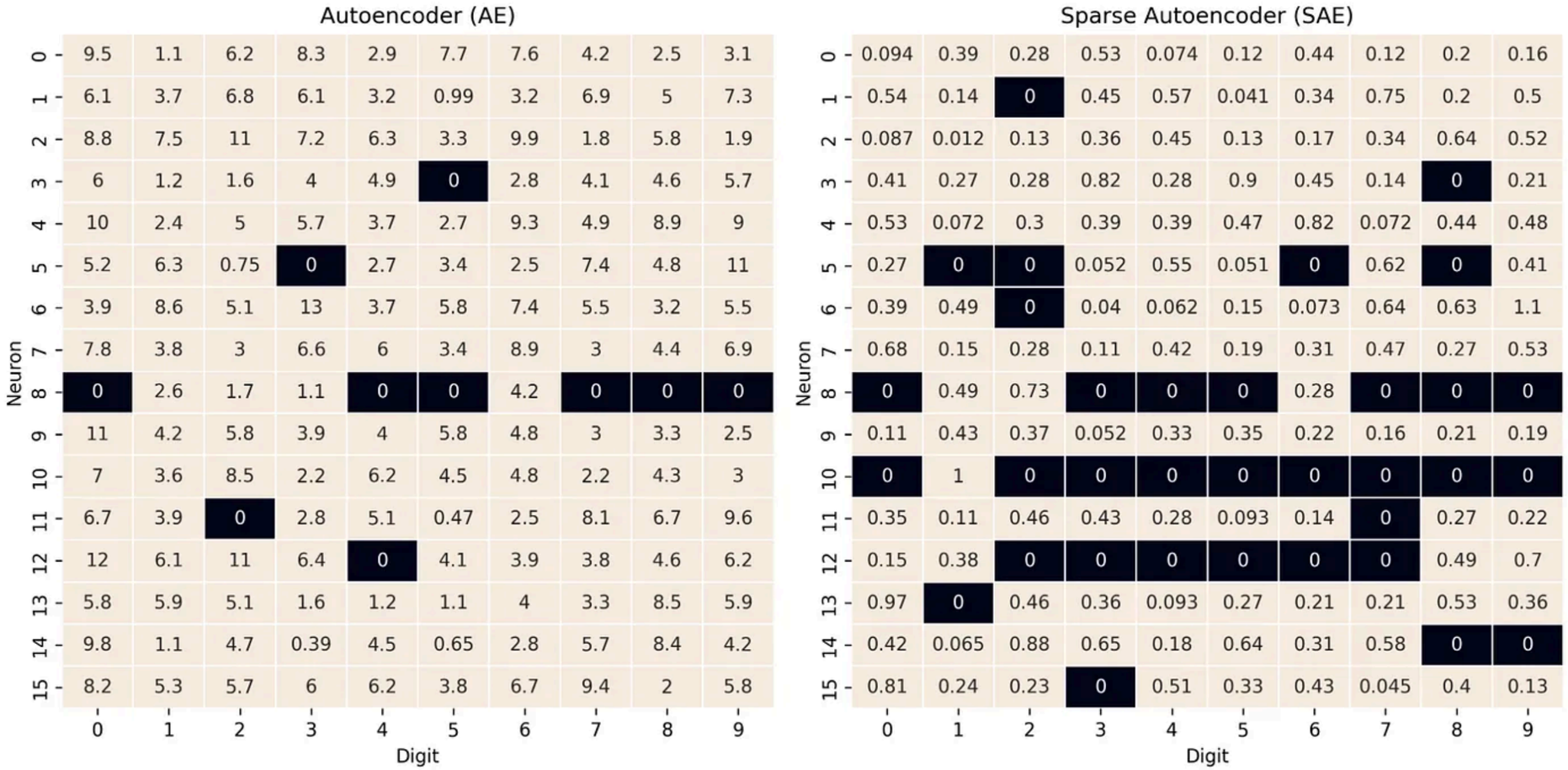
```python
def forward(self, x):
  encoded = self.encoder(x)
  decoded = self.decoder(encoded)
  return decoded, encoded
```

L1 Loss를 인코딩벡터에 붙였다?

“sparse하게 되으면 좋겠어..!!”



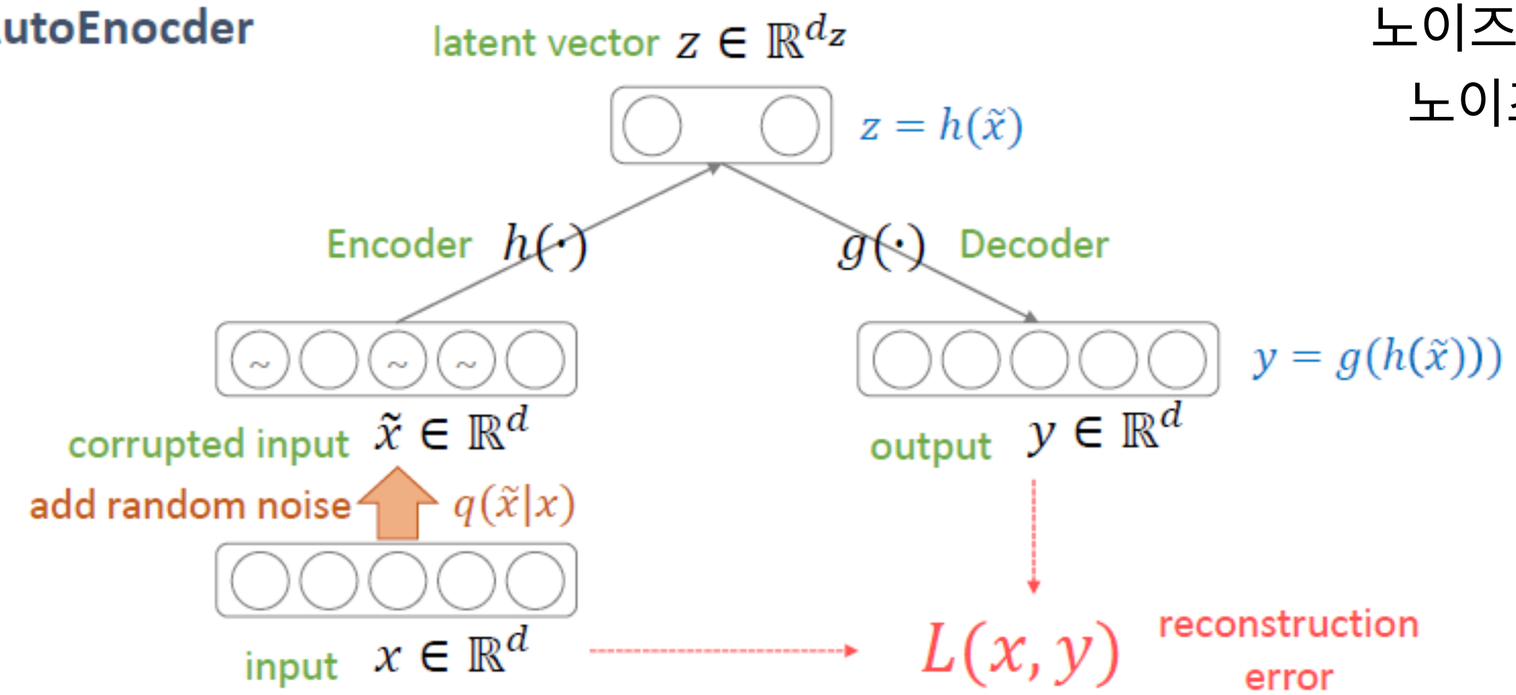
실제로 sparse 해지나!?



https://wikidocs.net/193829

Denoising Autoencoder

학습할때 노이즈를 끼움.
노이즈를 제거하는 용도
노이즈에 강하게 키움!



Minimize $L_{DAE} = \sum_{x \in D} E_{q(\tilde{x}|x)} [L(x, g(h(\tilde{x}))]$

http://videolectures.net/deeplearning2015_vincent_autoencoders/?q=vincent%20autoencoder

```python
# Add noise function
def add_noise(img, noise_factor=0.5):
  noisy_img = img + noise_factor * torch.randn_like(img)
  noisy_img = torch.clamp(noisy_img, 0., 1.)
  return noisy_img

# Initialize the Denoising Autoencoder, Loss function, and Optimizer
denoising_model = DenoisingAutoencoder()
criterion = nn.MSELoss()
optimizer = optim.Adam(denoising_model.parameters(), lr=0.001)
```

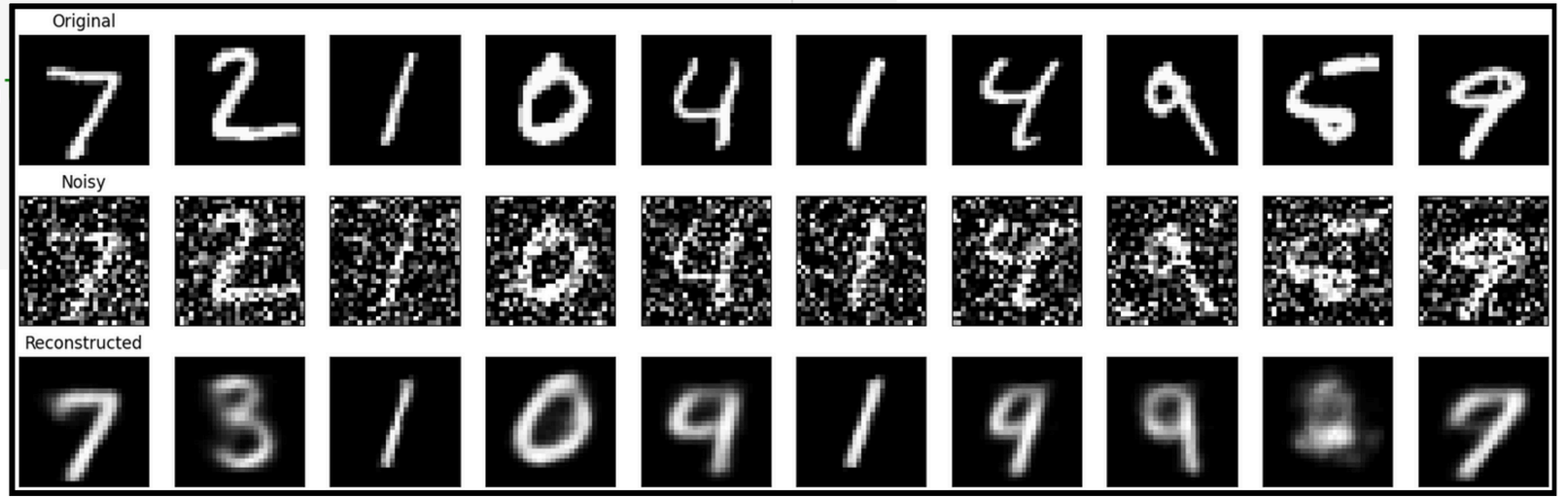
노이스를 더하자!!!

```python
for epoch in range(num_epochs):
  for data in train_loader:
    img, _ = data
    img = img.view(img.size(0), -1)  # Flatten the original images

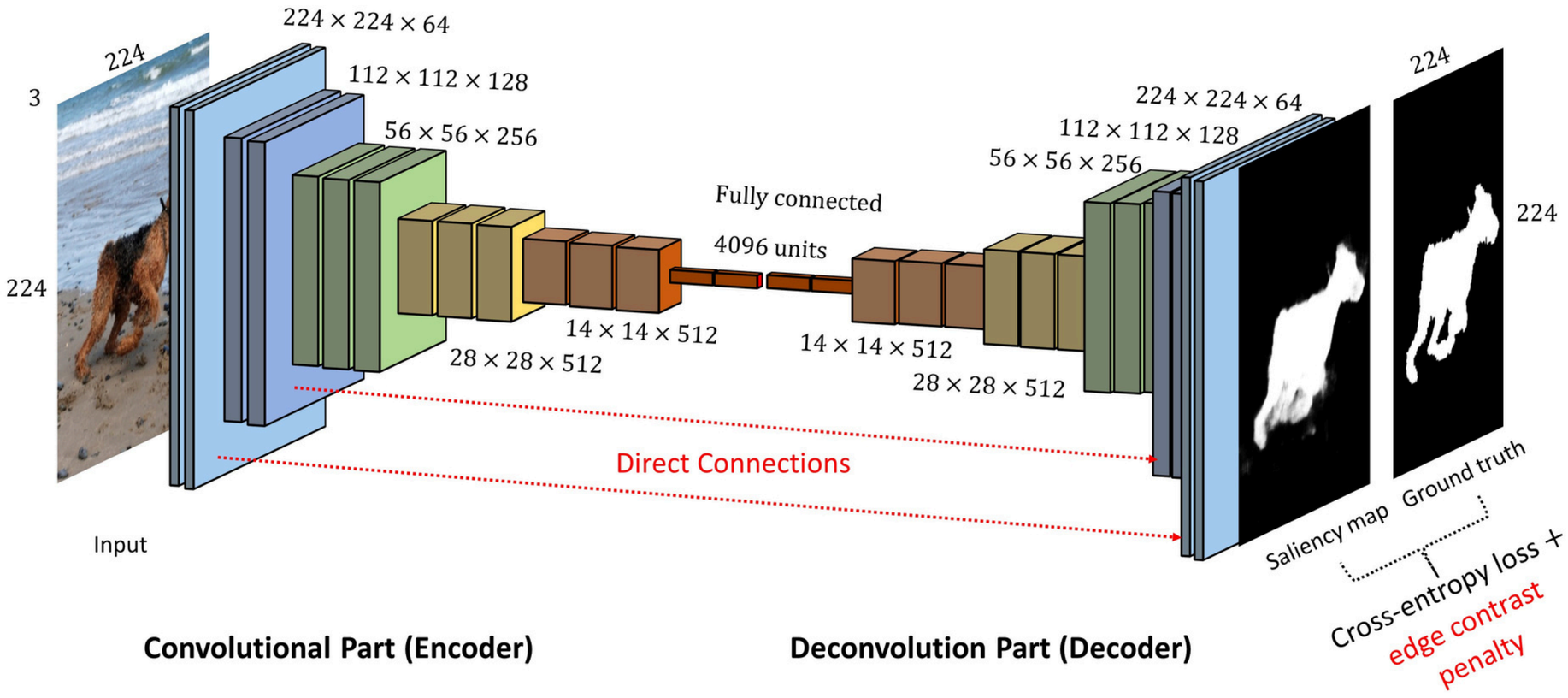
    # Add noise to the images
    noisy_img = add_noise(img)

    recon = denoising_model(noisy_img)
    loss = criterion(recon, img)  # Ca

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```



Convolutional Autoencoder



이미지니까 Convolutional layer!!!

```python
class ConvAutoencoder(nn.Module):
    def __init__(self):
        super(ConvAutoencoder, self).__init__()

        # Encoder layers
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), # (16, 14, 14)
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), # (32, 7, 7)
            nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=7) # (64, 1, 1) - Latent space
        )

        # Decoder layers
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, kernel_size=7), # (32, 7, 7)
            nn.ReLU(),
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=2, padding=1, output_padding=1), # (16, 14, 14)
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1), # (1, 28, 28)
            nn.Sigmoid() # Output pixel values are between 0 and 1
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```

```python
# Initialize the Convolutional Autoencoder, Loss function, and Optimizer
conv_model = ConvAutoencoder()
conv_criterion = nn.MSELoss()
conv_optimizer = optim.Adam(conv_model.parameters(), lr=0.001)

# Training loop for Convolutional Autoencoder
num_epochs = 20 # Can adjust this

print("\nStarting training for Convolutional Autoencoder...")

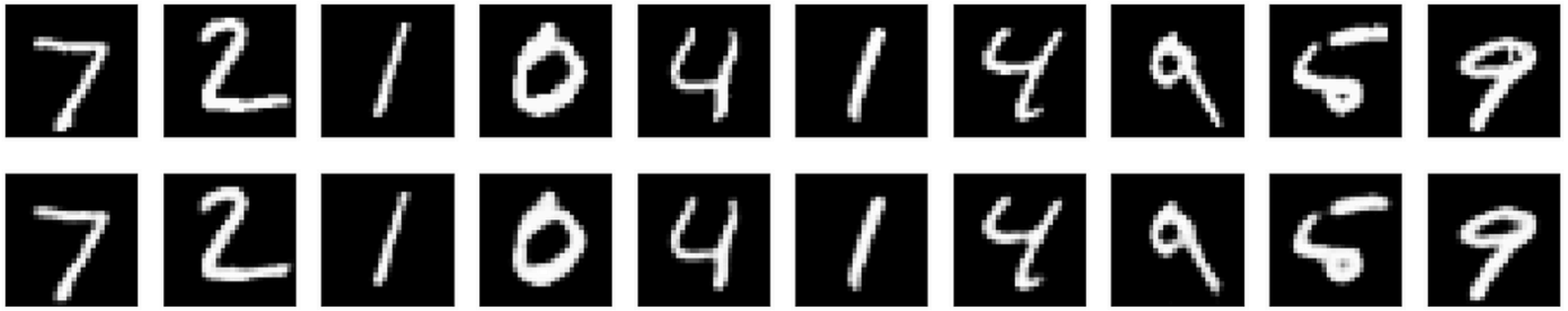
for epoch in range(num_epochs):
    for data in train_loader:
        img, _ = data
        # No need to flatten for Conv layers
        recon = conv_model(img)
        loss = conv_criterion(recon, img)

        conv_optimizer.zero_grad()
        loss.backward()
        conv_optimizer.step()
```

```python
# Visualize reconstruction results for Convolutional Autoencoder
conv_model.eval()
with torch.no_grad():
    dataiter = iter(test_loader)
    images, _ = next(dataiter)
    reconstructed_images = conv_model(images)

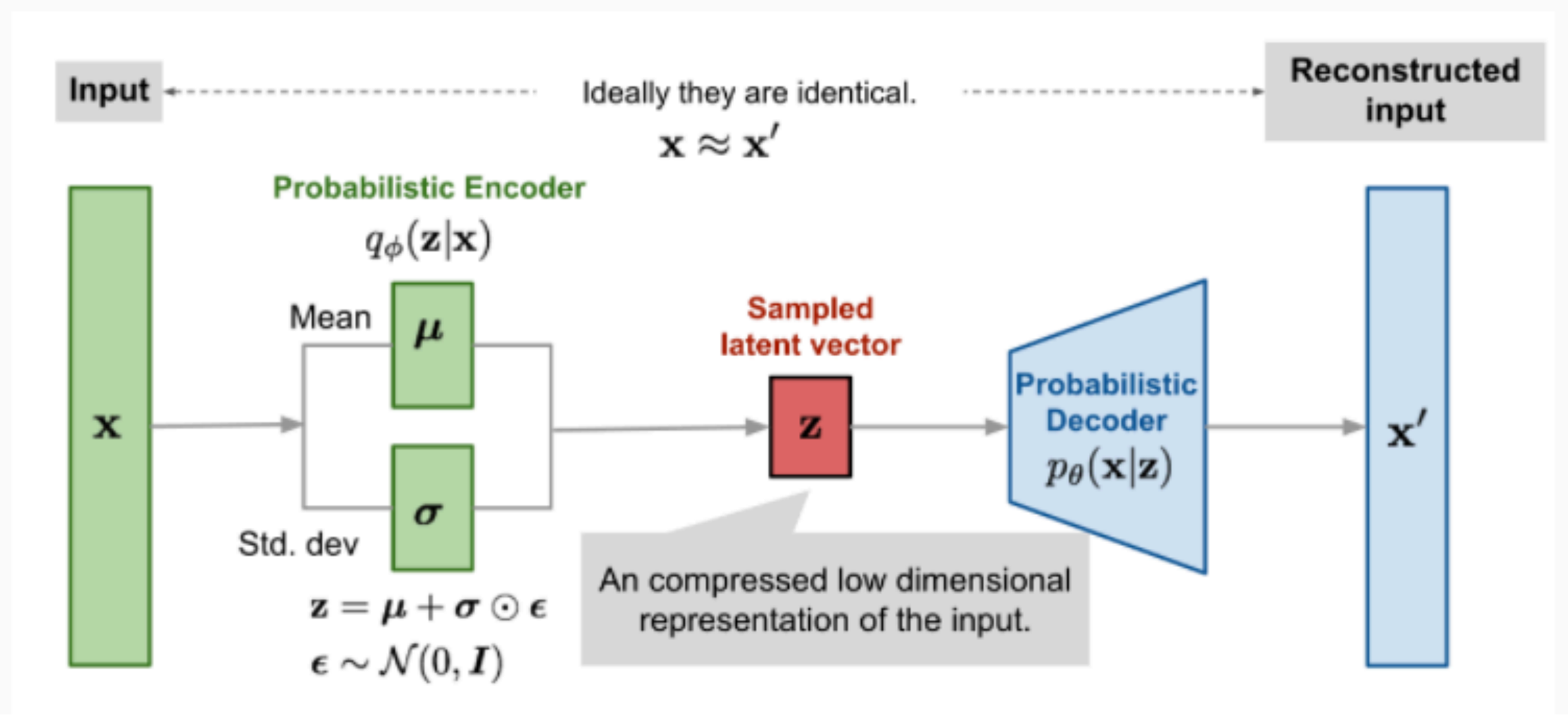
    n = 10
    plt.figure(figsize=(20, 4))
    for i in range(n):
        # Original Images
        ax = plt.subplot(2, n, i + 1)
        plt.imshow(images[i].squeeze().numpy(), cmap='gray') # Squeeze the channel dimension
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)

        # Reconstructed Images
        ax = plt.subplot(2, n, i + 1 + n)
        plt.imshow(reconstructed_images[i].squeeze().numpy(), cmap='gray') # Squeeze the channel dimension
        ax.get_xaxis().set_visible(False)
        ax.get_yaxis().set_visible(False)
    plt.show()
```



Variational Autoencoder

만약... 평균과 표준편차의 형태로 학습시킬수있다면 어떨까?



```python
class VAE(nn.Module):
  def __init__(self, latent_dim):
    super(VAE, self).__init__()

    self.fc1 = nn.Linear(28 * 28, 512)
    self.fc2_mu = nn.Linear(512, latent_dim)
    self.fc2_logvar = nn.Linear(512, latent_dim)
    self.fc3 = nn.Linear(latent_dim, 512)
    self.fc4 = nn.Linear(512, 28 * 28)

    self.relu = nn.ReLU()
    self.sigmoid = nn.Sigmoid()

  def encode(self, x):
    h1 = self.relu(self.fc1(x))
    return self.fc2_mu(h1), self.fc2_logvar(h1)

  def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std

  def decode(self, z):
    h3 = self.relu(self.fc3(z))
    return self.sigmoid(self.fc4(h3))

  def forward(self, x):
    mu, logvar = self.encode(x.view(-1, 28 * 28))
    z = self.reparameterize(mu, logvar)
    return self.decode(z), mu, logvar
```

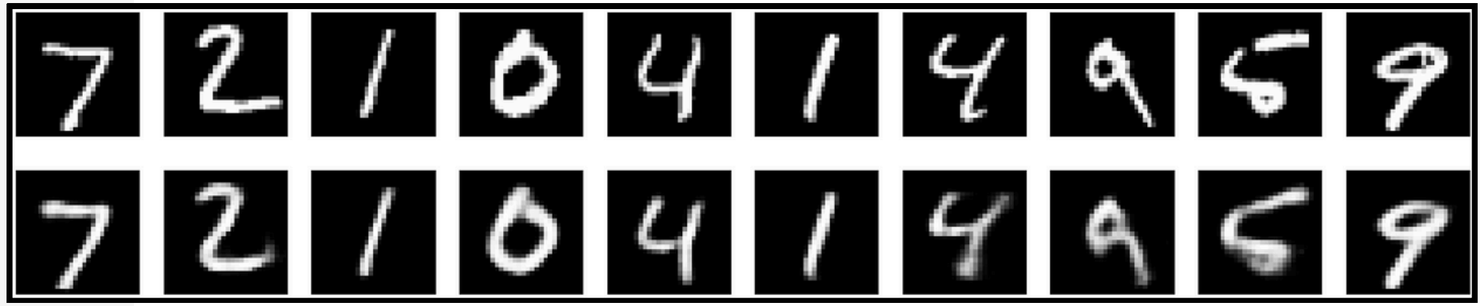
```python
# VAE Loss Function
def vae_loss_function(recon_x, x, mu, logvar):
  BCE = nn.functional.binary_cross_entropy(recon_x, x.view(-1, 28 * 28), reduction='sum')
  KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
  return BCE + KLD

# Initialize VAE
latent_dim = 20 # You can adjust the latent dimension
vae_model = VAE(latent_dim)
vae_optimizer = optim.Adam(vae_model.parameters(), lr=0.001)

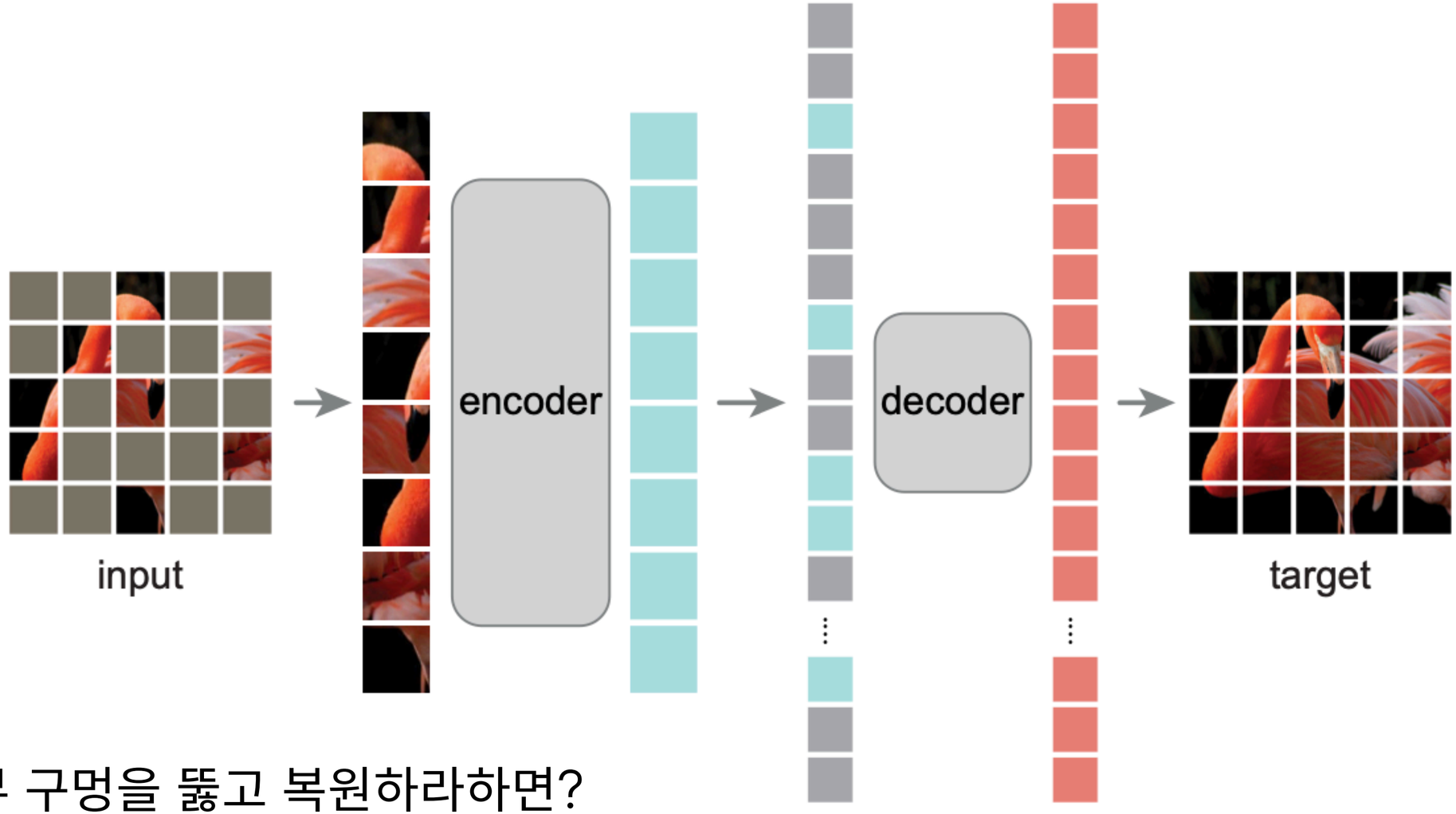
# Training loop for VAE
num_epochs = 20 # Can adjust this

print("\nStarting training for Variational Autoencoder...")

for epoch in range(num_epochs):
  vae_model.train()
  train_loss = 0
  for batch_idx, (data, _) in enumerate(train_loader):
    data = data.to(vae_model.fc1.weight.device) # Ensure data is on the same device as the model
    vae_optimizer.zero_grad()
    recon_batch, mu, logvar = vae_model(data)
    loss = vae_loss_function(recon_batch, data, mu, logvar)
    loss.backward()
    train_loss += loss.item()
    vae_optimizer.step()
```

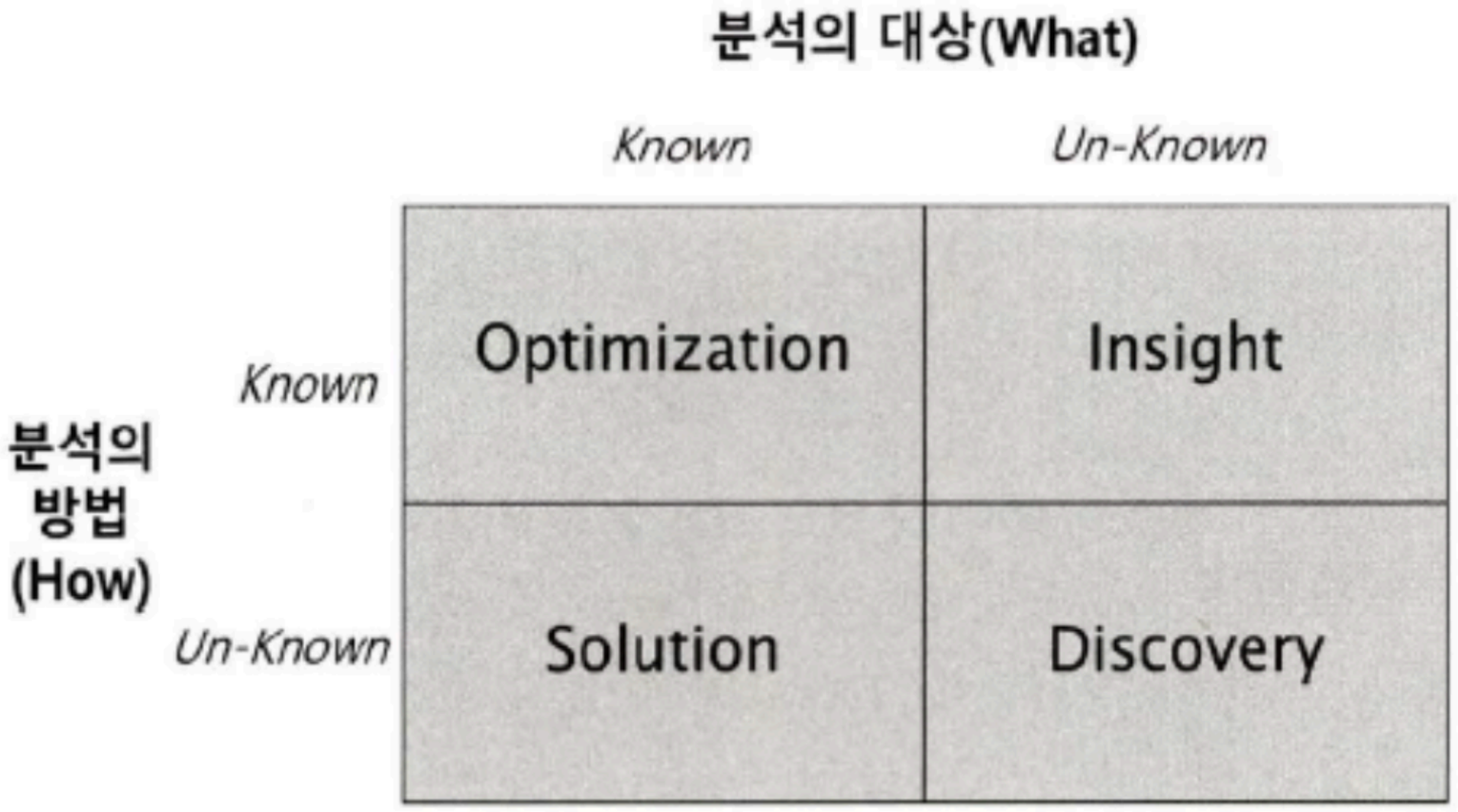


Masked autoencoder



이미지에 일부 구멍을 뚫고 복원하라하면?

더 많은 오토인코더가 있지요~


분석의 대상(What)
Known
Un-Known
분석의 방법 (How)
Known
Un-Known
Optimization
Insight
Solution
Discovery

여러분은 어떤 인공지능이 만들고 싶나요?

만들어보세요!